

← Go Back

Hardware

MKR IoT Carrier Rev2

Tutorials

MKR IoT Carrier Rev2
User Manual

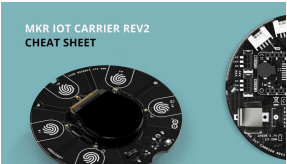


Home / Hardware / MKR IoT Carrier Rev2 / **MKR IoT Carrier Rev2 User Manual**

MKR IoT Carrier Rev2 User Manual

Learn how to set up the MKR IoT Carrier Rev2, get a quick overview of the compatible boards, components, and the library.

Author: Liam Aljundi, Karl Söderby
Last revision: 09/30/2025



The Arduino MKR IoT Carrier Rev2

The Arduino® MKR IoT Carrier Rev2 is equipped with different sensors, actuators and with a display to leave you free to focus on prototyping and programming your next IoT projects. The carrier does not come equipped with a microcontroller, meaning it only works in combination with an Arduino board. The MKR IoT Carrier Rev2 comes equipped with 5 RGB LEDs, 5 capacitive touch buttons, a colored display, IMU and a variety of quality sensors. It also features a battery holder for a 18650 Li-Ion battery, SD card holder and Grove connectors.

ON THIS PAGE

Compatible Boards	—
Assembly	
Datasheet	
Arduino Cloud	
Pinout	
Grove Connectors	
Carrier Library	—
Initialization	
Setup	
BME688 Environmental Sensor	
Code	
IMU Accelerometer & Gyroscope Sensors	+
RGB and Gesture Sensor	+
Relays	+
Peripherals	+
Memory	+
Power	

The MKR family of boards share a common pinout, meaning that they can all be easily connected to the MKR IoT Carrier Rev2. Here is a list of the MKR boards:

MKR 1000 WiFi

MKR FOX 1200

MKR GSM 1400

MKR NB 1500

MKR Vidor 4000

MKR WAN 1300

MKR WAN 1310

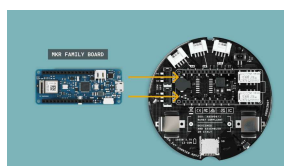
MKR WiFi 1010

MKR Zero

The MKR family boards offer different features that can be useful for your projects. You can [browse through the boards](#) and pick the one that is most compatible for your project.

Assembly

The chosen Arduino MKR board needs to be mounted on top of the MKR IoT Carrier Rev2 and connected to a computer. The board can be connected as shown in the image below, matching the pin numbers on the board to the ones on the carrier.



Mounting board on the
MKR IoT Carrier Rev2

The full datasheet is available as a downloadable PDF from the link below:

[Download the Arduino
MKR IoT Carrier Rev2
datasheet](#)

Arduino Cloud

The MKR IoT Carrier Rev2 can be controlled through the [Arduino Cloud](#), a Cloud service that allows us to create IoT applications in just minutes, if combined with an Arduino Cloud compatible board. Here's a list of the Arduino MKR family boards that are compatible with the Arduino Cloud:

[MKR 1000 WiFi](#)

[MKR GSM 1400](#)

[MKR NB 1500](#)

[MKR WAN 1300](#)

[MKR WAN 1310](#)

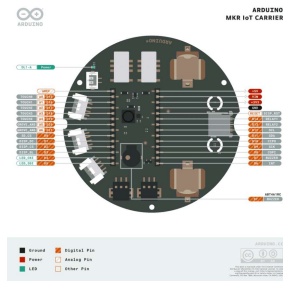
[MKR WiFi 1010](#)

Note: The MKR GSM 1400 and MKR NB 1500 require a SIM card to connect to the Cloud, as they communicate over the mobile networks.



The MKR WAN 1300 and 1310 board requires a Arduino PRO Gateway LoRa® to connect to the Cloud.

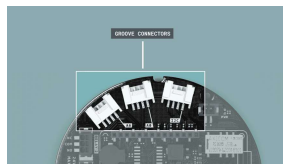
If you need help to get started, you can go through the [Arduino Cloud tutorial](#).



The pinout for MKR IoT
Carrier Rev2

The full pinout is available in the [resources section](#).

Grove Connectors



Grove connectors on
the MKR IoT Carrier
Rev2

The MKR IoT Carrier Rev2 comes with **three grove connectors** (2 analog and 1 I2C) that enables us to easily connect external sensors. The type of the connector is labeled clearly on the back of the carrier.

Carrier Library

To program the MKR IoT Carrier Rev2, the [Arduino_MKRIoTCarrier](#) library needs to be included. This library allows us to control and read all the components of the MKR IoT Carrier Rev2. Setting up

code lines in the **initialization** and **setup**. Practically speaking, the code used in the initialization and setup is rarely changed, and it is required in every sketch.

Initialization

In the initialization (the very top) of every sketch, the **Arduino_MKRIoTCarrier** library needs to be included, which includes the individual libraries of the components mounted onto the carrier.

Next, an object of type `MKRIoTCarrier` needs to be created. We'll call it `carrier` for simplicity.

```
1 #include <Arduino_MKRIoTCarrier.h>
2 MKRIoTCarrier carrier;
```

Setup

Inside the `setup()` of every sketch that created, 2 lines of code needs to added.

First, we need to tell the library whether the carrier is being used inside the **plastic enclosure** included in the Oplà IoT Kit or not. The library uses this information in order to calibrate the sensitivity of the capacitive buttons.

If you use the plastic enclosure, add this line:

```
1 carrier.withCase();
```

Otherwise, add this line:

```
1 carrier.noCase();
```

After that, the `carrier.begin();` command is needed. Final example is:

```
1 void setup() {  
2   carrier.noCase();  
3   carrier.begin();  
4 }
```

BME688 Environmental Sensor



The BME688 Humidity
Sensor on the MKR IoT
Carrier Rev2

The **BME688 Environmental Sensor** is a 4 in 1 sensor capable of providing the following measurements:

Humidity, 0-100% r.H.

Temperature, -40 to +85 °C.

BME688 is a low-power sensor with current consumption as low as **2.1 μ A at 1 Hz**. It communicates over the I²C bus as other sensors onboard this carrier.

Code

The values from the **temperature, humidity & pressure** can be retrieved and stored in **float** variables as shown below. For this, we need to use the `Env` class.

```
1 float temperature = car
2 float temperature = car
3 float humidity = carrie
4 float pressure = carrie
```



Temperature is returned in degrees Celsius, while relative humidity is returned in percentage.

To access **gas** various values, we need to use the `AirQuality` class.

```
1 float gasResistor = car
2 float volatileOrganicCo
3 float co2 = carrier.Air
```

IMU Accelerometer & Gyroscope

The IMU (Inertial Measurement Unit) used on the MKR IoT Carrier Rev2 is the located **behind the display**.

LSM6DSOX

The [LSM6DSOX](#) from STM is an IMU (Inertial Measurement Unit) that features a 3D digital **accelerometer** and a 3D digital **gyroscope**.

Code

The 3-axis values from the **accelerometer** and **gyroscope** sensors can be retrieved and stored in **float** variables as shown below:

```
1 float x, y, z;
2
3 void loop(){
4   if (carrier.IMUmodule
5     {
6       carrier.IMUmodule
7       Serial.println(x)
8     }
9 }
```

The following methods can be used to detect movement:

```
1 carrier.IMUmodule.accel
```

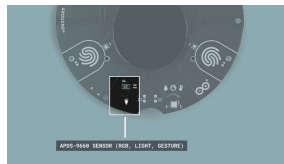
Returns 0 if no new acceleration data sample is available, 1 if new


```
1 carrier.IMUmodule.readA
```

Reads acceleration data from the sensor on the three axis and assigns it to the provided variables.

The underlying library used to read the sensor is [LSM6DSOX](#).

RGB and Gesture Sensor



The APDS-9960 sensor
on the MKR IoT Carrier
Rev2

The MKR IoT Carrier Rev2 contains a Broadcom **APDS-9960 RGB and Gesture sensors**, situated under the display and marked with a bulb icon. The sensor is useful for **ambient light** and **RGB** color sensing, **proximity** sensing, and **gesture** detection.

Code

The color values from the **RGB** sensor can be retrieved and stored in **int** variables as shown below.

The

method can be used to detect colors.

The if statement

```
if (r >= 135 && g >= 135 &&
    b >= 135)
```

in the code below checks if the color detected ranges from light gray to white, but the numbers assigned to red (r), green (g) and blue (b) can be customized to any desired color.

```
1  int r, g, b;
2
3  void loop(){
4
5      if (carrier.Light.readColor() >= 135 &&
6          carrier.Light.readColor() >= 135 &&
7              carrier.Light.readColor() >= 135)
8          // check if color is white
9          if (r >= 135 && g >= 135 && b >= 135)
10             Serial.print("White");
11         }
12     }
13 }
```

The following methods can be used to detect gesture:

```
1  carrier.Light.gestureAvailable()
```

Checks the availability of the **gesture** sensor.

```
1  carrier.Light.readGesture()
```

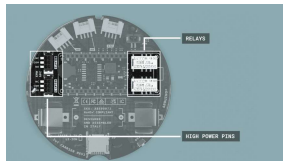
Confirms which gesture is being

The code example below shows the gesture value in a fixed width integer `uint8_t` variable, and the if statement checks `if (gesture == UP)`. To detect movements in other directions, UP can be replaced with DOWN, RIGHT or LEFT.

```
1 void loop(){
2
3     if (carrier.Light.
4     {
5         uint8_t gesture
6         Serial.print("Ge
7
8         // when gesture
9         if (gesture == U
10        {
11            Serial.println
12        }
13    }
```

The underlying library used to read the sensor is [Arduino_APDS9960](#).

Relays



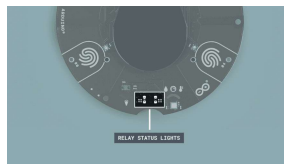
The relays on the MKR IoT Carrier Rev2

The MKR IoT Carrier Rev2 is equipped with two 5V Coil voltage **KEMET EE2-5NU-L relays**, located on the back side of the carrier. The relays are non-latching with a

contacts, and can take up a max of **2A** Current and **24 V** of input each.

The connections between a high power circuit and the relays will be done through these connectors (marked "**high power pins**" in the image above).

Once the cables are introduced inside the connectors, they will automatically be locked inside. To unlock a cable and remove it from the connector, a tool is needed to be inserted through the top square hole (it can be a flat screwdriver, a hard piece of plastic, etc.).



The L1 and L2 LEDs indicators

The **L1 and L2 LEDs** on the carrier are visual indicators of the state of the relays. If the LED is ON, it means that the **COM** and the **NO** terminal of the relay are **connected**, and if the LED is OFF it means that **COM** and **NC** are **connected**.

Code

The `carrier.RelayX.open();` and `carrier.RelayX.close();` methods can be used to control the relays, and `carrier.RelayX.getStatus();` to read the status of the relays.

Swap to the Normally Open (NO) circuit of relay 1 (turns it on):

```
1 carrier.Relay1.open();
```

Swap to the Normally Closed (NC) circuit of relay 2 (turns it off), default mode on power off:

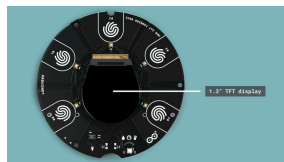
```
1 carrier.Relay2.close();
```

Bool, returns the status LOW means NC and HIGH means NO:

```
1 carrier.Relay2.getStatu
```

Peripherals

Display



The MKR IoT Carrier
Rev2's display

The screen on the MKR IoT Carrier Rev2 is a **rounded 1.3" TFT display**, with a 240 x 240 resolution and a diameter of 36 x 40 mm

Code

The display is controlled through the `carrier.display` object which is an instance of the **Adafruit_ST7789** class, based on the more general **Adafruit_GFX** interface. Most tutorials mentioning **Adafruit_GFX** should be usable on your MKR IoT Carrier Rev2.

To get started, check the [Adafruit_GFX documentation](#) and see the examples included in the `Arduino_MKTIoTCarrier` library.

] (<https://www.arduino.cc/reference/en/libraries/adafruit-st7735-and-st7789-library/>), which is included in the carrier's library, meaning that it does not need to be added it separately.

We'll list here some of the most useful methods to configure the MKR IoT Carrier Rev2's display, including basic configurations, background and text colors, font size, position of the cursor and a loading animation.

```
1 carrier.display.fillRect
```

This method sets the color of the background of the display using hex codes. Example values are:

`0xFFFF` for white

`0x0000` for black

`0xF800` for red

`0x07FF` for green

`0x07FF` for cyan`0xF81F` for magenta`0xFFE0` for yellow`0xFC00` for orange

```
1 carrier.display.setRotation(0)
```

This method sets the angle of the screen. 0 is the starting position with no rotation. The screen can only be rotated 90, 180 or 270 degrees by replacing the 0 with 1, 2 or 3.

```
1 carrier.display.drawBitmap(x, y, w, h, bitmap_visual)
```

This method displays a bitmap visual on the carrier's screen. The values x & y values are the top left coordinates where the visual is drawn, w & h are the width and height of the visual. The data for the bitmap_visual graphic can be stored as a byte array and used as the `bitmap_visual`.

```
1 carrier.display.setTextColor(0xFF0000)
```

This method sets the color of the text using hex codes.

```
1 carrier.display.setTextColor(0xFF0000)
```

This method sets the text size. 2 is an average text size since it is visible, but not too small, 3 is a bit larger.

```
1 carrier.display.setCursor
```

This method is very important, as it indicates where on the printing starts on the display. It is indicated by pixels, so, if **0, 0** are used for example, it will start printing in the top left corner.

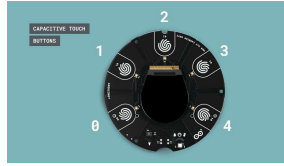
```
1 display.print("text");
```

This method will print the text inside the string at the current cursor position.

More Resources

In order to develop a graphical user interface with the MKR IoT Carrier Rev2, the [Arduino_OplaUI](#) library can be used. This library uses the color LEDs, the buzzer and the touch buttons to build an interactive user interface featuring multiple pages. It also includes a few predefined gauges to display values. See the library examples to get started.

Buttons



The MKR IoT Carrier
Rev2's buttons

The carrier has five **capacitive touch buttons** on its top side, numbered from 0 to 4. The buttons are sensitive to direct touch and can also detect wireless touch.

Code

```
1 carrier.Buttons.update(
```

Reads the state of the pads and save them to be used in the different types of touch events.

```
1 carrier.Buttons.getTouc
```

Get if the pad is getting touched, true until it gets released.

```
1 carrier.Buttons.onTouch
```

Get when have been a touch down.

```
1 carrier.Buttons.onTouch
```

Get when the button has been released.

```
1 carrier.Buttons.onTouch
```

Get both, touched and released.

Replace `TOUCH0` with `TOUCH1`, `TOUCH2`, `TOUCH3`, `TOUCH4` in the examples above to access the other buttons. The code example below shows how the status of a button can be checked.

```
1 void loop(){
2   // updates buttons
3   carrier.Buttons.up
4
5   // Checks if button
6   if (carrier.Button
7   {
8     Serial.print("Bu
9   }
10 }
```

LEDs



The LEDs on the MKR IoT Carrier Rev2

the top side of the carrier in front of the buttons.

Code

The LEDs are controlled with the Adafruit® DotStar library, which is included in the `MKRIoTCarrier` library.

The `carrier.leds.show();` method is necessary for updating the new state of the LEDs and needs to be called after any change of the state of the LEDs (turning on & off or change of color).

Here are some of the useful methods used to control the LEDs on the MKR IoT Carrier Rev2:

```
1 carrier.leds.setPixelCo
```

Sets the color of the index's LED.

```
1 carrier.leds.setBrightn
```

Set the overall brightness, from 0 (no brightness) to 255 (maximum brightness).

```
1 carrier.leds.clear();
```

Clear the buffer of the LEDs.

```
1 carrier.leds.fill(color
```

Fill X amount of the LEDs with the same color.

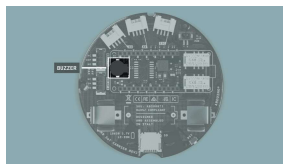
```
1 uint32_t myColor = carr
```

Saves a custom color.

The code example below shows how to light up all 5 LEDs with our customized color.

```
1 #include <Arduino_MKRI
2 MKRIoTCarrier carrier;
3
4 uint32_t myCustomColor
5
6 void setup() {
7     carrier.noCase();
8     carrier.begin();
9     carrier.leds.fill(m
10    carrier.leds.show()
11 }
```

Buzzer



The buzzer on the MKR
IoT Carrier Rev2

The MKR IoT Carrier Rev2 is equipped with a **sound buzzer** on the back of the carrier.

Code

The buzzer can be controlled with the following methods:

```
1 carrier.Buzzer.sound(fr
```

Equivalent to `tone()`, it will make the tone with the selected frequency.

```
1 carrier.Buzzer.noSound(
```

Equivalent to `noTone()`, it will stop the tone signal.

```
1 carrier.Buzzer.beep();  
2 carrier.Buzzer.beep(800
```

This method is a handy shortcut generating a beep. The two arguments are optional and can be set to customize the frequency and the duration (in milliseconds);

Memory

The MKR IoT Carrier Rev2 contains a **SD card slot** that accepts a Micro SD.

Code

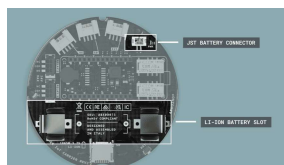
already included in the
`MKRIoTCarrier` library.

The SD class initialized in the main
`carrier.begin()` so you don't
need to do it yourself. The code
below demonstrates how to save
data on a file on a SD card.

```
1  #include <Arduino_MKRI  
2  MKRIoTCarrier carrier;  
3  
4  File myFile;  
5  
6  void setup() {  
7      carrier.noCase();  
8      carrier.begin(); /  
9  
10     myFile = SD.open("t  
11 }
```

In order to learn more, check any
of the many tutorials about using
the `SD` library on Arduino.

Power

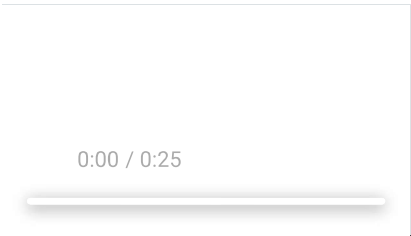


JST battery connector
on the MKR IoT Carrier
Rev2.

The MKR IoT Carrier Rev2 can be
either powered through a USB
cable connected to the mounted
MKR board, or through a battery.
The battery used should be a LI-
ION 18650 3.7v battery, which can

battery holder on the bottom side.

In order to use the USB power to charge the battery, a little cable with JST connectors on both ends is needed between the MKR IoT Carrier Rev2 and the MKR board. The battery can then be recharged via a USB connection through the MKR Board (runs up to 48h with a 3.7v 2500mAh).



Suggest changes

The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page [here](#).

Need support?

[Help Center](#)
[Ask the Arduino Forum](#)
[Discover Arduino](#)
[Discord](#)

License

The Arduino documentation is licensed under the [Creative Commons Attribution-Share Alike 4.0](#) license.

Was this article helpful?



